

# Uniform Backward Pass Optimization Summary

gpurec profiling notes

May 2026

## 1 Scope and headline results

This note summarizes the optimization work documented in `docs/uniform-backward-fp32-fused-profile.m` and `docs/uniform-backward-50tree-wave2-profile.md`. The focus is the CUDA uniform backward path. The tables use the benchmark timings recorded in those documents; when a gain is called “incremental”, it is the adjacent comparison from the same benchmark run, not a mathematically additive global attribution.

The high-level result is:

| Benchmark             | Early baseline | Final documented default | Gain                     |
|-----------------------|----------------|--------------------------|--------------------------|
| 10 families           | 73.252 ms      | 37.204 ms                | 36.048 ms faster, 49.2%  |
| 50 families           | 230.795 ms     | 121.016 ms               | 109.779 ms faster, 47.6% |
| 50-family peak memory | 16.806 GB      | about 10.308 GB          | 6.498 GB lower, 38.7%    |

Most wins came from reducing memory traffic, removing scratch tensors, and removing host synchronization points. The hot kernels were not limited by matrix multiply throughput. Nsight Compute repeatedly showed memory bandwidth, long-scoreboard stalls, barriers, and register pressure as the relevant constraints.

## 2 Uniform backward dataflow

Each backward wave has three important families of work:

1. The self-loop or wave kernel, `_wave_backward_uniform_kernel`. It applies the fixed-point Neumann VJP for the wave and accumulates parameter contributions.
2. The DTS split work. The backward pass recomputes DTS forward quantities for split waves and then runs `_dts_cross_backward_accum_kernel` to add direct `Pi` adjoints and produce the adjoints needed by the uniform `Pibar` VJP.
3. The uniform `Pibar` VJP tree kernel. It propagates split-side `Pibar` adjoints through the species-tree correction.

Before the later optimizations, a 50-family Nsight Systems capture broke the kernel work down as follows:

| Component                                         | Time      | Share of kernel time |
|---------------------------------------------------|-----------|----------------------|
| <code>_wave_backward_uniform_kernel</code>        | 39.578 ms | 27.7%                |
| <code>_uniform_cross_pibar_vjp_tree_kernel</code> | 37.954 ms | 26.6%                |
| <code>_dts_cross_backward_accum_kernel</code>     | 29.678 ms | 20.8%                |
| <code>_dts_fused_kernel</code>                    | 12.136 ms | 8.5%                 |
| PyTorch float reductions                          | 5.498 ms  | 3.9%                 |
| PyTorch active-mask abs/max work                  | 7.910 ms  | 5.5%                 |

The optimization strategy was therefore to keep the fused Triton/CUDA path, but remove avoidable dense intermediates, redundant passes, host scalar reads, and scratch-buffer round trips.

### 3 Optimization timeline

| Change                                   | Local comparison                             |               | 10-family gain | 50-family gain                    | Why it helped                                                                              |
|------------------------------------------|----------------------------------------------|---------------|----------------|-----------------------------------|--------------------------------------------------------------------------------------------|
| Leaf species index                       | 73.252<br>68.373 ms<br>230.795<br>225.373 ms | →<br>and<br>→ | 4.879 ms       | 5.422 ms                          | Avoided dense $[W, S]$ leaf tensors and their reads.                                       |
| Read-only RHS, phase 1                   | 68.373<br>67.984 ms<br>225.373<br>218.372 ms | →<br>and<br>→ | 0.389 ms       | 7.001 ms                          | Made the fused wave kernel treat the RHS as read-only, reducing snapshot/copy pressure.    |
| Empty spec scratch                       | 67.984<br>68.160 ms<br>218.372<br>214.755 ms | →<br>and<br>→ | neutral at 10  | 3.617 ms                          | Allocated scratch with <b>empty</b> instead of zeroing buffers that the kernel overwrites. |
| Direct global reductions                 | 68.160<br>65.428 ms<br>214.755<br>205.489 ms | →<br>and<br>→ | 2.732 ms       | 9.266 ms                          | For $G = 1$ , replaced indexed <b>scatter_add</b> style reductions with direct sums/adds.  |
| Fused wave parameter accumulation        | 65.428<br>60.614 ms<br>205.489<br>181.570 ms | →<br>and<br>→ | 4.814 ms       | 23.919 ms                         | Removed six full $[W, S]$ contribution tensors and their PyTorch reductions.               |
| Default <code>max_wave_size=32768</code> | 60.614<br>68.095 ms<br>181.570<br>181.255 ms | →<br>and<br>→ | neutral        | 0.315 ms plus 3.774 GB lower peak | Capped wave batches to keep large runs under memory limits with little speed cost.         |
| Compact Pibar scratch                    | 61.095<br>59.462 ms<br>181.255<br>174.174 ms | →<br>and<br>→ | 1.633 ms       | 7.081 ms                          | Stored one compact coefficient instead of three scratch arrays.                            |

| Change                                                         | Local comparison |                                                   | 10-family gain | 50-family gain                    | Why it helped                                                                                                              |
|----------------------------------------------------------------|------------------|---------------------------------------------------|----------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| RHS view, final per-wave clone removal                         | 48.993           | →                                                 | 0.653 ms       | 3.980 ms                          | Removed exactly one device-to-device copy per processed wave; 50-family D2D bytes fell from 2609.790 MB to 35.638 MB.      |
|                                                                | 48.340 ms        | and                                               |                |                                   |                                                                                                                            |
|                                                                | 163.035          | →                                                 |                |                                   |                                                                                                                            |
|                                                                | 159.055 ms       |                                                   |                |                                   |                                                                                                                            |
| Triton active-mask builder                                     | 157.820          | →                                                 | about 0.8 ms   | 4.9–5.9 ms                        | Replaced PyTorch <code>abs().max()</code> plus compare kernels with one row-reduction Triton kernel.                       |
|                                                                | 151.909 ms       | at 50 families in the final default-flip check    |                |                                   |                                                                                                                            |
| Merged DTS S-term accumulation                                 | 153.198          | →                                                 | about 1.2 ms   | 3.1–4.6 ms                        | Reused values already live in the DTS species loop instead of making a second species pass.                                |
|                                                                | 148.625 ms       | at 50 families in the final forced-old comparison |                |                                   |                                                                                                                            |
| Device-resident DTS scalar parameters                          | about 149.6      | →                                                 | about 2.5 ms   | 1.0–1.5 ms                        | Removed 66 CUDA scalar-to-Python extractions in the 50-family split waves.                                                 |
|                                                                | 148.4 ms         | at 50 families                                    |                |                                   |                                                                                                                            |
| In-kernel DTS scalar reduction accumulation                    | 148.449          | →                                                 | neutral        | about 0.9 ms mean, median neutral | Removed small PyTorch scalar reductions for large split waves; kept vector <code>grad_mt</code> accumulation disabled.     |
|                                                                | 147.563 ms       | mean at 50 families                               |                |                                   |                                                                                                                            |
| DTS-staged Pibar VJP                                           | 44.802           | →                                                 | 4.191 ms       | 11.501 ms                         | DTS writes <code>u.d</code> and row sums directly, so the tree VJP skips denominator construction and large reads.         |
|                                                                | 40.611 ms        | and                                               |                |                                   |                                                                                                                            |
|                                                                | 148.387          | →                                                 |                |                                   |                                                                                                                            |
|                                                                | 136.886 ms       |                                                   |                |                                   |                                                                                                                            |
| Wave speciation gather, int32 topology, 8-warps, row-max reuse | 40.050           | →                                                 | 2.846 ms       | 15.143 ms                         | Replaced zero/scatter/readback scratch traffic with parent gathers, shrank topology loads, and used the best launch shape. |
|                                                                | 37.204 ms        | and                                               |                |                                   |                                                                                                                            |
|                                                                | 136.159          | →                                                 |                |                                   |                                                                                                                            |
|                                                                | 121.016 ms       |                                                   |                |                                   |                                                                                                                            |

## 4 Instruction-level changes

### 4.1 Leaf species index

The old path materialized a dense leaf contribution over all wave rows and species. The new path carries a compact leaf-species index per clade row and lets each Triton program decide whether its species lane is the leaf species.

```
# old idea
leaf_term = full((W, S), -inf)
leaf_term[row, leaf_species[row]] = log_pS[leaf_species[row]]
term = combine(term, leaf_term)

# new idea inside the wave kernel
s_leaf = leaf_species_index[row]
leaf_value = where(s == s_leaf, log_pS[s], -inf)
term = combine(term, leaf_value)
```

This helped because the representative wave kernels were bandwidth-bound. The optimization removed dense  $[W, S]$  allocation and read traffic for data that is mostly  $-\infty$ .

### 4.2 Read-only RHS and final RHS view

The critical correctness fact is that the fused wave kernel only reads the current wave RHS. Later DTS and Pibar adjoints write to earlier child rows, not to the current wave slice. Therefore the current slice does not need a clone.

```
# old
rhs_k = accumulated_rhs[ws:we].clone()
wave_backward(rhs_k, ...)

# new
rhs_k = accumulated_rhs[ws:we]      # read-only view
wave_backward(rhs_k, ...)
```

The final view change removed 45 D2D copies for 10 families and 49 D2D copies for 50 families. At 50 families, D2D copy time fell from 5.159 ms to 0.391 ms and D2D bytes fell by 2574.152 MB.

### 4.3 Fused wave parameter accumulation

The old path wrote multiple full contribution matrices and reduced them later with PyTorch. The fused path accumulates the relevant species contribution inside `_wave_backward_uniform_kernel`.

```
# old
aw0, aw1, aw2, aw3, aw4, aw5 = wave_kernel_outputs(...)
grad_E += (aw0 + aw2).sum(dim=0)
grad_D += aw3.sum(dim=0)
grad_S += aw4.sum(dim=0)
grad_T += aw5.sum(dim=0)

# new, in the Triton program
local_E = aw0_s + aw2_s
local_D = aw3_s
local_S = aw4_s
local_T = aw5_s
tl.atomic_add(grad_E + s, local_E)
tl.atomic_add(grad_D + s, local_D)
```

```
tl.atomic_add(grad_S + s, local_S)
tl.atomic_add(grad_T + s, local_T)
```

NCU showed global RED operations after this change, but the tradeoff was favorable: the largest leaf wave removed roughly 1.82 GB of DRAM traffic and the 50-family wall time improved by 23.919 ms in that optimization ladder.

## 4.4 Compact Pibar scratch

The old Neumann scratch stored several arrays:

```
# old scratch
pibar_wt
inv_denom
p_prime
```

The compact path stores the coefficient that is actually needed later and recomputes the cheap normalized probability when needed:

```
# new scratch
pibar_coeff = pibar_wt * inv_denom

# later
u_d = term * pibar_coeff
p_prime = exp2(Pi - row_max)
```

The largest profiled 10-family leaf wave went from 4.032 GB to 2.694 GB of total DRAM traffic after compact scratch. Occupancy stayed about the same, so the win came from issuing fewer memory transactions, not from more resident work.

## 4.5 Kernelized active-mask construction

The original mask construction used several PyTorch kernels:

```
clade_max = rhs_k.abs().max(dim=1).values
active_mask = clade_max >= pruning_threshold
```

The accepted version uses a Triton row-reduction kernel:

```
active_mask = active_mask_from_rhs_absmax(rhs_k, pruning_threshold)
```

This does not remove the host pruning decisions. The Python loop still uses `any()`, `sum().item()`, and `nonzero()` to decide which waves and rows to process. It helped anyway because it replaced about 8.1 ms of PyTorch active-mask GPU work with about 2.95 ms of Triton work in the 50-family profile.

## 4.6 Merged DTS S-term accumulation

The old DTS accumulation kernel made two species passes. The second pass existed mostly for the S term and reloaded values that had already been touched.

```
# old
for s in species:
    compute D term
    compute T terms
    write grad_Pibar_l/r
    accumulate scalar params

for s in species:
```

```

    reload Pi_l, Pi_r, parent Pi, v_k
    compute S term
    atomic_add accumulated_rhs

# new
for s in species:
    compute D term
    compute T terms
    compute S term while masks and values are live
    write grad_Pibar_l/r
    accumulate scalar params and direct Pi adjoints

```

NCU confirmed the mechanism: on the largest DTS launch, total DRAM bytes fell from 3.644 GB to 3.345 GB and excessive global sectors fell by about 132 MB. The cost was higher register pressure, from 54 to 78 registers per thread, and lower occupancy. The net effect was still positive: the DTS accumulation bucket fell by 3.535 ms in Nsight Systems.

## 4.7 DTS scalar parameters stay on device

The old wrappers converted CUDA scalar tensors into Python floats before each Triton launch:

```

# old
pD_val = float(log_pD.item())
pS_val = float(log_pS.item())
dts_kernel(..., pD_val, pS_val)

# new
dts_kernel(..., log_pD_device_ptr, log_pS_device_ptr)

# inside the kernel
log_pD = tl.load(log_pD_ptr)
log_pS = tl.load(log_pS_ptr)

```

For the 50-family workload there are 33 split waves, hence 66 scalar extractions for `log_pD` and `log_pS`. Nsight Systems measured exactly 66 fewer `cudaStreamSynchronize` calls, 66 fewer `cudaMemcpyAsync` calls, and 66 fewer D2H copy events. The DTS kernel itself did not get faster; the end-to-end interval got faster because the host no longer stopped to read those scalars.

## 4.8 DTS scalar reduction accumulation

The scalar-parameter path used to materialize per-split arrays and then reduce them:

```

# old
param_pD, param_pS = dts_backward(...)
grad_log_pD += param_pD.sum()
grad_log_pS += param_pS.sum()

```

The promoted path accumulates only the two scalar parameter gradients inside large split waves:

```

# new inside the DTS accumulation kernel
sum_pD = reduce_over_species(local_pD_contrib)
sum_pS = reduce_over_species(local_pS_contrib)
tl.atomic_add(grad_log_pD, sum_pD)
tl.atomic_add(grad_log_pS, sum_pS)

```

This was a small cleanup, not a major kernel optimization. It removed 36 tiny PyTorch launches in the 50-family Nsight capture and improved the 50-family mean by about 0.9 ms,

while the median was essentially neutral. The `grad_mt` vector accumulation variant was implemented but left default-off because it introduced many atomics into only  $S = 1999$  species targets and made the register-sensitive DTS kernel slower.

#### 4.9 DTS-staged Pibar VJP

This was the largest split-side improvement. The old path materialized `grad_Pibar_l/r`; the tree VJP then read those matrices, computed `u_d`, wrote `u_d` into a subtree buffer, and finally performed the tree correction.

```
# old
grad_Pibar_l, grad_Pibar_r = dts_backward_accum(...)

for side in (left, right):
    inv_denom = compute_denominator(Pi_child, mt)
    u_d = grad_Pibar_side * inv_denom
    subtree_buf = u_d
    pibar_tree_vjp(subtree_buf)
```

The new path computes the exact input needed by the tree VJP in the DTS kernel:

```
# new
# DTS already has vd1/vd2, where:
#   grad_Pibar_l = vd2
#   grad_Pibar_r = vd1

inv_denom = exp2(row_max + mt - Pibar_star)
u_d = vd * inv_denom
A = sum_s(u_d[s])

write u_d_staging
write A
pibar_tree_vjp_from_ud(u_d_staging, A)
```

The stable identity

$$\text{inv\_denom}_{r,s} = 2^{\text{row\_max}_r + \text{mt}_s - \text{Pibar}_{r,s}}$$

allows the DTS kernel to use forward-stored row maxima and the already materialized `Pibar` value.

The tradeoff was clear in the profiles. DTS accumulation became slower (26.237 ms  $\rightarrow$  30.006 ms in Nsight) because it writes the staged `u_d`, stores row sums, and carries more live values. But the `Pibar` tree bucket fell from 37.888 ms to 25.510 ms, and PyTorch float reductions fell from 5.357 ms to 0.369 ms. NCU showed the tree kernel instructions falling from 3.085 billion to 0.900 billion, registers per thread falling from 56 to 40, and achieved occupancy rising from 74.58% to 98.37%.

#### 4.10 Wave speciation gather, int32 topology, and row-max reuse

After the staged `Pibar` work, the wave kernel became the largest bucket again. Source counters showed scratch traffic around Neumann speciation as the main issue. The old code effectively did this:

```
# old scatter-style speciation
zero(spec_buf_next)

for parent in wave_rows:
    spec_buf_next[child1[parent], :] += term[parent, :] * sl1[parent, :]
    spec_buf_next[child2[parent], :] += term[parent, :] * sl2[parent, :]
```

```

for child in wave_rows:
    spec = spec_buf_next[child, :]
    next_term[child, :] = combine(spec, ...)

```

The new path asks each child row to gather from its parent:

```

# new gather-style speciation
parent = parent_index[child]
side_weight = where(child == child1[parent], s11[parent, :], s12[parent, :])
spec = term[parent, :] * side_weight
next_term[child, :] = combine(spec, ...)

```

This adds parent-index loads but removes full-row zeroing, scatter stores, and readback. The same change also made 32-bit topology useful: species-tree indices fit in int32, so the kernel avoids repeated 64-bit topology loads. The winning launch shape for this gather path was `num_warps=8`; larger `BLOCK_S` and 16 warps were slower.

Representative NCU numbers for the large wave kernel:

| Metric                | Old scatter | Final default |
|-----------------------|-------------|---------------|
| Duration              | 7.847 ms    | 5.102 ms      |
| DRAM read             | 2.941 GB    | 1.008 GB      |
| DRAM write            | 2.437 GB    | 1.895 GB      |
| L1 global load bytes  | 40.352 GB   | 20.259 GB     |
| L1 global store bytes | 6.600 GB    | 3.518 GB      |
| L1 global RED bytes   | 2.638 GB    | 1.327 GB      |
| Achieved occupancy    | 82.51%      | 99.43%        |
| Registers/thread      | 48          | 40            |
| Excessive L2 sectors  | 777.6 MB    | 146.1 MB      |

The final wave-kernel proposal moved the 50-family median from about 136.2 ms to about 121.0 ms. Nsight Systems attributed almost all of the gain to `_wave_backward_uniform_kernel`, whose bucket fell from 39.422 ms to 25.412 ms.

## 5 Rejected or diagnostic experiments

Several optimizations were implemented or prototyped but deliberately not promoted. They are useful because they explain where not to spend the next engineering pass.

| Experiment                    | Result                                                                                                                             | Interpretation                                                                                                                  |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Disable pruning               | 50-family time<br>230.795 ms →<br>324.348 ms in the<br>early table; later<br>157.329 ms →<br>241.924 ms.                           | Pruning still skips too much real work to remove it wholesale.                                                                  |
| Fixed-schedule device pruning | 50-family median<br>about 174.387 ms →<br>177.588–177.943 ms<br>in the first profile;<br>later 157.329 ms →<br>161.084–161.736 ms. | It removed D2H syncs, but launched kernels for waves the host-pruned path skipped. No-op scheduling cost exceeded sync savings. |



| Experiment                          | Result                                                                                                        | Interpretation                                                                                              |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Pruning threshold $10^{-8}$         | 230.795 ms $\rightarrow$ 218.066 ms from the early baseline, but slower than the accepted path.               | More aggressive thresholding was not the right source of speedup and risked changing pruning behavior.      |
| Child-grouped uniform Pibar VJP     | Tree VJP bucket 37.784 ms $\rightarrow$ 32.119 ms, but group-reduce cost was 16.102 ms; net path 48.221 ms.   | Duplicate split-side factor was only about 1.111 $\times$ , too small to pay for a separate reduction pass. |
| Forward Pibar stat reuse alone      | 10-family median 47.769 ms $\rightarrow$ 47.290 ms, but 50-family median 162.704 ms $\rightarrow$ 166.956 ms. | It reduced ancestor-gather work but added global Pibar/mt reads; at 50 families the DRAM pressure won.      |
| Backward row-stat precompute        | Tree VJP improved by 0.848 ms, but a 2.690 ms precompute pass was added.                                      | A full extra pass over row data was more expensive than the saved denominator work.                         |
| Species-tree prefix denominator     | 50-family median 162.704 ms $\rightarrow$ 180.791 ms; Pibar VJP bucket 37.784 ms $\rightarrow$ 56.475 ms.     | It reduced some gather stalls but introduced level barriers and more L2/DRAM traffic.                       |
| Grouped DTS backward accumulation   | 50-family median 164.352 ms $\rightarrow$ 175.860 ms.                                                         | High-fanout waves were not child-duplicate-heavy; grouping added materialization and reductions.            |
| No-atomic DTS accumulation          | 50-family median 164.352 ms $\rightarrow$ 163.053 ms, but Nsys DTS bucket was unchanged.                      | Correct for child-unique cases, but atomics were not the dominant cost in this layout.                      |
| Disable kernelized backward DTS     | 50-family time 180.732 ms $\rightarrow$ 649.828 ms in the early comparison.                                   | The fused DTS backward path is essential.                                                                   |
| No-split final VJP shortcut         | 50-family 181–184 ms range, slower than compact scratch.                                                      | Algebra was valid, but it read scratch and increased DRAM traffic.                                          |
| Recompute Pibar denominator scratch | 50-family 174.754 ms, slightly worse than compact coefficient scratch 174.174 ms.                             | Diagnostic useful for understanding traffic, but compact scratch was better.                                |

| Experiment                           |          | Result                                                                     | Interpretation                                                                                                                 |
|--------------------------------------|----------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Leaf-logp only/scalar specialization | hit-spe- | 50-family 182.300 ms, worse than the accepted path.                        | Leaf log-probability loads were not the limiting traffic after leaf indexing.                                                  |
| Block/warp sweep for wave kernel     |          | BLOCK_S=128: 143.905 ms;<br>BLOCK_S=512: 137.661 ms; 16 warps: 125.623 ms. | The winning point was BLOCK_S=256, 8 warps on the gather path. Larger shapes increased loop trips, pressure, or locality cost. |

## 6 Why the accepted changes worked

The successful changes fall into four categories.

### 6.1 Fewer bytes

This was the dominant theme. Leaf indexing, compact `Pibar` scratch, RHS view mode, staged `Pibar` VJP, and wave speciation gather all removed large global-memory streams. The clearest examples are:

- compact `Pibar` scratch removed about 1.34 GB of DRAM traffic from one representative 10-family leaf-wave launch;
- RHS view removed 2574.152 MB of D2D copies at 50 families;
- staged `Pibar` VJP reduced representative tree-kernel DRAM read by about 681 MB and cut executed instructions by about 71%;
- wave gather reduced representative wave-kernel DRAM read by about 1.93 GB and excessive L2 sectors by about 81%.

### 6.2 Fewer launches and PyTorch reductions

Fused wave parameter accumulation, direct global reductions, DTS scalar reduction accumulation, and staged `Pibar` VJP removed PyTorch reduction kernels from the hot path. Some of these were small individually, but they also removed large temporary tensors. Proposal 5, for example, reduced total kernel launches from 3100 to 2968 in its Nsight capture and cut PyTorch float-sum reductions from 5.357 ms to 0.369 ms.

### 6.3 Fewer host synchronizations

Device-resident DTS scalars did not change the bulk GPU kernel profile. Its value was removing 66 scalar D2H reads and their stream synchronizations. Kernelized active-mask construction did not remove the host pruning decisions, but it did remove several PyTorch kernels used to build the mask.

### 6.4 Better data layout

The wave gather path and int32 topology change are layout optimizations. The important change was not more arithmetic parallelism; it was changing the memory access pattern so the kernel no longer writes a full speciation scratch buffer only to read it back. Int32 topology then reduced the width of repeated index loads.

## 7 Correctness discipline

The reports consistently paired performance work with parity checks:

- old-vs-new loss and theta-gradient comparisons on 3, 10, and 50 family workloads;
- focused Triton kernel tests for active mask, DTS backward accumulation, uniform cross-Pibar VJP, staged DTS+Pibar, and scatter-vs- gather wave behavior;
- finite-difference/autograd bridge tests, including forced fallback paths through environment variables;
- fp32/fp64 direct parity for the DTS scalar, reduction, and staged Pibar paths where applicable.

Small fp32 differences were accepted only where the summation order changed, for example the fused parameter accumulation path. The documented maximum relative differences were on the order expected from reordered fp32 reductions, and the finite-difference bridge tests passed.

## 8 Current bottleneck after these changes

After proposal 6, the wave kernel is no longer the dominant bucket. The next large costs are again split-side work: DTS accumulation and the staged Pibar tree VJP. The successful changes also make clear what future work should avoid. A monolithic DTS+Pibar kernel is unattractive because the staged DTS kernel already rose to 96 registers per thread. Device pruning must preserve the useful scheduling property of host pruning; simply launching all waves with active masks was slower.

The next likely direction is a device-resident active worklist or a more parent-aware DTS layout: preserve pruning’s ability to skip work, avoid host scalar decisions, and reduce the high-fanout split-wave traffic without adding large materialized reduction passes.